FUNCTIONAL REQUIREMENTS FOR THE NYXCIPHER WEBSITE & SMART CONTRACT (ERC20)

VERSION HISTORY				
VERSION	APPROVED BY	REVISION DATE	DESCRIPTION OF CHANGE	AUTHOR
1.0.0	Durchdenwald Schulze	07/30/2024	Draft	Daniel Cooper

1.0 INTRODUCTION

1.1 Purpose

The purpose of this document is to outline the functional requirements for the upgrade of NyxCipher.ai from version 1.0 to version 2.0.
This upgrade is a strategic initiative aimed at significantly enhancing the platform's capabilities in several key areas, including cryptocurrency analysis, scam detection, and market sentiment analysis.
By leveraging advanced technologies and methodologies, NyxCipher.ai version 2.0 will provide users with more accurate, timely, and comprehensive insights, thereby empowering them to make more informed decisions in the dynamic and often volatile cryptocurrency market.

Background

NyxCipher.ai was initially developed as a cutting-edge tool to assist users in navigating the complex world of cryptocurrency.
Version 1.0 successfully established a foundation by offering basic functionalities such as real-time data access, preliminary scam detection, and rudimentary market sentiment analysis.
However, the rapidly evolving nature of the cryptocurrency landscape, coupled with increasing user demands for more sophisticated tools, necessitates a substantial upgrade to the platform.

Objectives

The primary objectives of the NyxCipher.ai version 2.0 upgrade are as follows:

- Enhance Analytical Capabilities: Introduce advanced algorithms and machine learning models to improve the accuracy and depth of cryptocurrency analysis.

- Improve Scam Detection: Develop more robust mechanisms for identifying and flagging potential scams, thereby protecting users from fraudulent activities.

- Refine Market Sentiment Analysis: Utilize natural language processing (NLP) and other techniques to provide more nuanced and actionable insights into market sentiment.

- Optimize User Experience: Redesign the user interface to be more intuitive and user-friendly, ensuring that users can easily access and interpret the information they need.

- Strengthen Security: Implement state-of-the-art security measures to safeguard user data and ensure the integrity of the platform.

Scope

This document will detail the functional requirements necessary to achieve the aforementioned objectives.
It will cover various aspects of the web application, including usability, performance, supportability, and security requirements.
Each section will provide specific, actionable requirements that will guide the development and implementation of NyxCipher.ai version 2.0.

Stakeholders

The primary stakeholders for this upgrade include:
• End Users: Individuals and organizations who rely on NyxCipher.ai for cryptocurrency analysis and decision-making.

• Development Team: Engineers and developers responsible for implementing the new features and improvements.

• Product Management: Team members who oversee the project, ensuring that it aligns with business goals and user needs.

• Security Experts: Professionals tasked with ensuring that the platform meets the highest standards of data protection and cybersecurity.

Benefits

The upgrade to NyxCipher.ai version 2.0 is expected to deliver several key benefits:
• Increased Accuracy: More precise analysis and insights will enable users to make better-informed decisions.

• Enhanced Security: Improved scam detection and robust security measures will protect users from potential threats.

• Better User Experience: A more intuitive interface and enhanced usability will make the platform more accessible and enjoyable to use.

• Scalability: The upgraded platform will be better equipped to handle a growing user base and increasing data volumes.

2.0 BUSINESS REQUIREMENTS OVERVIEW

The business requirements for NyxCipher.ai version 2.0 are centered around enhancing the platform's overall functionality and user experience.
This upgrade is designed to address the evolving needs of users and the dynamic nature of the cryptocurrency market.
The key areas of focus include improving user experience, expanding analytical capabilities, and ensuring robust security measures.
Below is a detailed overview of the business requirements for NyxCipher.ai version 2.0.

Enhanced Data Processing and Analysis for Real-Time Insights

One of the primary goals of the NyxCipher.ai version 2.0 upgrade is to enhance the platform's data processing and analysis capabilities.
This involves the integration of advanced algorithms and machine learning models to provide real-time insights into the cryptocurrency market.
The enhanced data processing will enable users to:

- Access Real-Time Data: Users will have access to up-to-the-minute data on various cryptocurrencies, allowing them to make timely and informed decisions.

- Perform In-Depth Analysis: The platform will offer more sophisticated analytical tools, enabling users to conduct comprehensive analyses of market trends, price movements, and trading volumes.

- Identify Patterns and Trends: Advanced machine learning models will help identify patterns and trends in the data, providing users with actionable insights that can inform their trading strategies.

Improved User Interface for Better Usability

A key aspect of the upgrade is the improvement of the user interface (UI) to enhance usability. The goal is to create an intuitive and user-friendly interface that makes it easy for users to navigate the platform and access the information they need. The improved UI will include:

- Streamlined Navigation: A more organized and logical layout that allows users to quickly find the features and tools they need.

- Enhanced Visual Design: A modern and visually appealing design that improves the overall user experience.

- Customizable Dashboards: Users will have the ability to customize their dashboards to display the information that is most relevant to them.

• Interactive Charts and Graphs: Enhanced visualizations that allow users to interact with the data and gain deeper insights.

Advanced Security Features to Protect User Data and Transactions

Security is a top priority for NyxCipher.ai version 2.0. The upgrade will introduce advanced security features to protect user data and transactions from potential threats. These security measures will include:

• Data Encryption: Robust encryption protocols to ensure that user data is protected both at rest and in transit.

• Multi-Factor Authentication (MFA): An additional layer of security that requires users to verify their identity through multiple methods.

• Regular Security Audits: Routine security assessments to identify and address vulnerabilities in the system.

• Compliance with Regulations: Adherence to relevant data protection regulations, such as GDPR and CCPA, to ensure that user data is handled responsibly.

Scalable Architecture to Support Growing User Base and Data Volume

As the user base of NyxCipher.ai continues to grow, it is essential to have a scalable architecture that can handle increased demand and data volume. The upgrade will focus on building a robust and scalable infrastructure that can support:

• Increased User Traffic: The platform will be designed to handle a higher number of concurrent users without compromising performance.

• Larger Data Sets: The architecture will be capable of processing and storing larger volumes of data, ensuring that users have access to comprehensive and up-to-date information.

• Seamless Integration: The system will be built with modular components that allow for easy integration of new features and third-party services.

3.0 FUNCTIONAL REQUIREMENTS IN WEB APP

3.1 Usability Requirements

Intuitive User Interface

The user interface (UI) of NyxCipher.ai version 2.0 must be designed to be intuitive and easy to navigate. This involves creating a layout that is logical and user-friendly, with clear instructions and tooltips to guide users through various features and functionalities. The UI should minimize the learning curve for new users while providing advanced options for experienced users. Key elements include:

- Consistent Design Language: Use a consistent design language across the platform to ensure a cohesive look and feel.

- Clear Visual Hierarchy: Organize content in a way that highlights the most important information and actions.

- Feedback Mechanisms: Provide immediate feedback for user actions, such as button clicks and form submissions, to enhance interactivity.

Multi-Language Support

To cater to a global user base, the platform must support multiple languages. This includes not only the translation of text but also the adaptation of cultural nuances and local conventions. The language selection should be easily accessible, allowing users to switch languages seamlessly. Considerations include:

- Localization: Adapt the platform to different regions, including date formats, currency symbols, and cultural references.

- Translation Accuracy: Ensure high-quality translations by using professional translation services and regularly updating language packs.

- User Preferences: Allow users to set and save their preferred language settings.

Responsive Design

The platform must employ a responsive design to ensure compatibility with various devices, including desktops, tablets, and smartphones. This involves using flexible layouts, images, and CSS media queries to create a consistent and optimal viewing experience across different screen sizes and orientations. Specific requirements include:

- Fluid Grid Layouts: Use fluid grid layouts that adjust to different screen sizes and orientations.

- Scalable Images: Implement scalable images that resize appropriately without losing quality.

• Touch-Friendly Elements: Design touch-friendly elements for mobile and tablet users, ensuring that buttons and links are easily tappable.

Accessibility Features

NyxCipher.ai version 2.0 must include accessibility features to support users with disabilities. This includes compatibility with screen readers, keyboard navigation, and other assistive technologies. The platform should adhere to the Web Content Accessibility Guidelines (WCAG) to ensure that all users can access and use the platform effectively. Key features include:
• ARIA Landmarks: Use ARIA landmarks to define regions of the page and improve navigation for screen reader users.

• Keyboard Shortcuts: Implement keyboard shortcuts for common actions to enhance accessibility for users who rely on keyboard navigation.

• Contrast Ratios: Ensure sufficient contrast ratios between text and background colors to improve readability for users with visual impairments.

3.2 Performance Requirements

High Data Volume Handling

The system must be capable of handling high volumes of data and user requests without significant delays. This involves optimizing database queries, using efficient algorithms, and implementing caching mechanisms to improve performance. The platform should be stress-tested to ensure it can handle peak loads. Specific strategies include:
• Database Indexing: Use indexing to speed up database queries and improve data retrieval times.

• Load Balancing: Implement load balancing to distribute user requests evenly across servers and prevent bottlenecks.

• Caching Strategies: Use caching strategies, such as in-memory caching and content delivery networks (CDNs), to reduce server load and improve response times.

Real-Time Data Processing

Real-time data processing and analysis are critical for providing timely insights to users. The system should be designed to process and analyze data within acceptable time frames, typically under 2 seconds for most operations. This requires the use of advanced data processing techniques and scalable infrastructure. Key components include:
• Stream Processing: Use stream processing frameworks to handle real-time data ingestion and analysis.

• Distributed Computing: Leverage distributed computing resources to parallelize data processing tasks and reduce latency.

• Efficient Algorithms: Implement efficient algorithms for data analysis to ensure quick and accurate results.

High Availability

The platform must maintain high availability to ensure that users can access the system whenever needed. The target uptime should be 99.9%, which translates to less than 9 hours of downtime per year. This involves implementing redundancy, failover mechanisms, and regular maintenance schedules to minimize downtime. Specific measures include:
• Redundant Systems: Use redundant systems and failover mechanisms to ensure continuous operation in case of hardware or software failures.

• Regular Backups: Perform regular backups of critical data to prevent data loss and facilitate quick recovery.

• Maintenance Windows: Schedule maintenance windows during off-peak hours to minimize disruption to users.

3.3 Supportability Requirements

Easy Maintenance and Updates

The system should be designed for easy maintenance and updates. This includes clear and comprehensive documentation for developers, outlining the architecture, codebase, and deployment procedures. The platform should support automated testing and continuous integration/continuous deployment (CI/CD) pipelines to streamline the update process. Key practices include:
• Code Documentation: Maintain thorough code documentation to help developers understand and modify the codebase.

• Automated Testing: Implement automated testing frameworks to ensure code quality and catch bugs early in the development process.

• CI/CD Pipelines: Use CI/CD pipelines to automate the build, test, and deployment processes, reducing the risk of errors and speeding up release cycles.

Modular Architecture

A modular architecture is essential for allowing seamless integration of new features and third-party services. This involves designing the system with loosely coupled components that can be independently developed, tested, and deployed. The use of microservices or service-oriented architecture (SOA) can facilitate this modular approach. Specific considerations include:
• Service Isolation: Design services to be isolated and independent, allowing them to be developed and deployed separately.

• API-First Approach: Use an API-first approach to define clear interfaces between services and ensure interoperability.

• Scalability: Ensure that each service can be scaled independently to handle varying loads and demands.

Comprehensive Logging and Monitoring

To facilitate troubleshooting and performance optimization, the platform must include comprehensive logging and monitoring capabilities. This involves capturing detailed logs of system events, user actions, and errors. Monitoring tools should provide real-time insights into system performance, resource usage, and potential issues, enabling proactive maintenance and quick resolution of problems. Key components include:
• Centralized Logging: Use centralized logging solutions to aggregate logs from different services and systems.

• Real-Time Monitoring: Implement real-time monitoring tools to track system performance and detect anomalies.

• AlertingMechanisms: Set up alerting mechanisms to notify administrators of critical issues and potential problems.

3.4 Security Requirements

Robust Data Encryption

Implementing robust encryption for data at rest and in transit is crucial to protect user information. This involves using industry-standard encryption algorithms and protocols, such as AES-256 for data storage and TLS 1.2 or higher for data transmission. Encryption keys should be managed securely, with regular rotation and access controls. Specific measures include:
• Encryption Standards: Adhere to industry standards for encryption to ensure data security and compliance.

• Key Management: Implement secure key management practices, including key rotation and access controls, to protect encryption keys.

• Data Masking: Use data masking techniques to protect sensitive information in non-production environments.

Multi-Factor Authentication (MFA)

To enhance account security, the platform must support multi-factor authentication (MFA). This requires users to verify their identity through multiple methods, such as a password and a one-time code sent to their mobile device. MFA helps prevent unauthorized access even if a user's password is compromised. Key features include:

• Authentication Methods: Support various authentication methods, including SMS, email, and authenticator apps, to provide flexibility for users.

• User Enrollment: Provide a straightforward enrollment process for users to set up and manage their MFA settings.

• Fallback Options: Offer fallback options, such as backup codes, to ensure users can access their accounts if they lose access to their primary authentication method.

Regular Security Audits

Regular security audits and vulnerability assessments are essential to identify and mitigate potential threats. This involves conducting periodic penetration testing, code reviews, and security assessments to uncover vulnerabilities. Any identified issues should be promptly addressed, and security patches should be applied regularly. Specific practices include:
• Penetration Testing: Conduct regular penetration testing to identify and address security weaknesses.

• Code Reviews: Perform thorough code reviews to ensure that security best practices are followed during development.

• Vulnerability Scanning: Use automated vulnerability scanning tools to detect and remediate security issues in the codebase and infrastructure.

Compliance with Data Protection Regulations

The platform must comply with relevant data protection regulations, such as the General Data Protection Regulation (GDPR) and the California Consumer Privacy Act (CCPA). This includes implementing measures to protect user privacy, such as data minimization, user consent, and the right to access and delete personal data. Compliance with these regulations helps build user trust and avoid legal penalties. Key requirements include:
• Data Minimization: Collect and process only the data necessary for the platform's functionality, reducing the risk of data breaches.

• User Consent: Obtain explicit user consent for data collection and processing activities, and provide clear information about how data will be used.

• Data Access and Deletion: Implement mechanisms for users to access, correct, and delete their personal data in accordance with regulatory requirements.

4.0 FUNCTIONAL REQUIREMENTS IN TOKEN DEVELOP

4.1 Functional Specifications

These functional specification cover requirements for the following categories:

1. General Implementation
2. Token Distribution & Tokenomics
3. Presale & Sale
4. Charities & Business
5. Costs
6. Access Control
7. Security & Testing
8. Deployment & Release Management
9. Advanced Features

4.2 General Token Implementation

GI-1	The contract should be a ERC20 token. It should implement this standard
GI-2	The contract should be able to dynamically change the fee by calling a method
GI-3	The contract should be able to restore the original fee rate set at initial deployment of the contract
GI-4	The contract should be able to determine the allowance of a specified address (Allowance is a reference to a required method in the ERC20 interface specification here. It is used to determine how much an address can spend from the total supply of tokens)
GI-5	The contract should be able to determine the available amount of tokens
GI-6	The contract should be able to determine the current transaction fees rate
GI-7	The contract should be able to exclude certain addresses from paying transfer fees
GI-8	The contract should be able to determine its name
GI-9	The contract should be able to determine its symbol
GI-10	The contract should implement safe transaction operations i.e safe mathematical calculations. In the event that an audited library does not yet exist to provide safe operations for ERC20 tokens the way that OpenZeppelin does for ERC20 tokens, then the contract should implement a local interface for safe transaction operations
GI-11	The contract should ensure that transactions fail if there is an insufficient balance
GI-12	The contract should ensure that transactions succeed if all conditions for the transactions are met

4.3 Token Distribution & Tokenomics

TDT-1	The contract should have an initial max supply of 1 Quadrillion tokens (1,000,000,000,000,000).
TDT-2	The liquidity pool of the contract should equal 40% of the max supply of tokens.
TDT-3	The dev & marketing wallet should equal 10% of the max supply of tokens.
TDT-4	The presale wallet should equal 10% of the max supply of tokens.
TDT-5	The contract should implement the following transaction fees:
	1. ANTI-RUG, TRANSACTION FEE: 20% Transaction fee for any token not held for at least 1 year in a wallet. To promote HODL. 2. NORMAL TRANSACTION FEE: 12% Transaction fee a. 1% to Marketing and Operations b. 5% to Charity Wallets c. 3.5% to Liquidity Pool d. 2.5% to a monthly lottery program 3. AUTO DONATION FEATURE: 50% of transaction fees will be automatically distributed to Charity Wallet selected by Wallet Owner.
TDT-6	The charity wallets should equal 20% of the max supply of available tokens
TDT-7	The founders wallet should equal 20% of the max supply of tokens.

4.4 Presale & Sale

PS-1	DxSale protocol should be used to lock creator wallets for some specified amounts of time
PS-2	Some locks will be gradually released over time so the token needs to account for that, and for different time lock release schedules per address if necessary (See email discussion)
PS-3	Tokens should be listed in DxSale for fair launch. Fair Launch tokens are tokens that have a publicly announced launch without any form of pre-mine. (see definition here of fair launch)

4.5 Charities & Business

CB-1	The contract should be able to send tokens directly to the address of a charity organization
CB-2	The contract should be able to remove fees associated with a particular transaction for special circumstances
CB-3	The contract should be able to integrate with internal and 3rd party APIs. For instance, it could integrate with an API for online based stores like our market place and OpenSea to facilitate the automatic transfer of transaction fees to a charity selected by the business owner

4.6 Costs

COS-1	The contract should employ best practices for solidity i.e optimal usage of storage and memory, arithmetic operations, mapping over arrays to eliminate the need for loops etc, in order to keep costs down
COS-2	The contract should be able to determine the current balance of a specified address
COS-3	The contract should be able to charge arbitrary fees. This might mean circumventing the recommended transfer utilities for one that can safely transfer tokens while charging the required transaction fee
COS-4	The contract should be able to revert failed transactions completely without any cost to the user

4.7 Access Control

AC-1	Contracts should be Ownable. The owner of a contract should be the account that deployed it
AC-2	The contract should be able to transfer ownership of the contract to a different account. Calling this function should be restricted to only the current owner of the contract being transferred
AC-3	The contract should be able to renounce ownership of the contract. Calling this function should be restricted to only the current owner of the contract being transferred.

4.8 Security & Testing

ST-1	There should be adequate unit testing to ensure that the contract works as expected
ST-2	There should be unit test that attempt to replicate network conditions like latency
ST-3	There should be unit tests to verify restrictions in the contract like access control
ST-4	The contract should use reusable modifiers to implement requirements for its restrictions
ST-5	The contract should be able to determine if it is communicating with another contract, an address or some other API
ST-6	The contract along with the rest of the project should follow security best practices i.e not exposing sensitive information in the contracts or the source code repository, using the most restrictive access modifiers possible for methods and variables in the

	contract(s) and taking all necessary precautions when integrating with 3rd party APIs etc. This requirement is especially important and must be included in the test suites/cases for the project.

4.9 Deployment & Release Management

DRM-1	The contract should be verifiable on ETHScan
DRM-2	The contract should be compatible with Defi exchange DApps like Pancake swap and Uniswap

4.10 Advanced Features

AF-1	The contract should be able to implement a daily lottery system that will reward holders with AirDrops. Holders should be notified if they win. The more of the token that is assigned to an address and the longer the tokens are held, the more chance the holder has of winning. 1% collected across all transactions for a day is given to a random address once a day
AF-2	The token should be able to facilitate the initialization of an ERC 721 NFT token for each donation to a charity organization.
AF-3	The ERC 721 token will be a completely different contract that the token will eventually communicate with
AF-4	The NFTs identifier will be a unique tokenID (ERC-721) that will be decryptable via a dApp. This dApp will be able to tell from the TokenID - the Time when the donation was made, the amount donated in Dollars, and the charity wallet that received the donation